A Unified Ingestion-to-Delivery Pipeline for Converting Multi-View 2D Media into 3D Assets

Charanya R
School of Computer Science and Engineering and Information Systems
Vellore Institute of Technology
Vellore, India
charanya.r@vit.ac.in

Priya M
School of Computer Science and Engineering and Information Systems
Vellore Institute of Technology
Vellore, India
priya.m@vit.ac.in

Siddhant Kishor Jain
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India
siddhj2206@gmail.com

Ayush Kumar
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India
theofficialayush.kumar@gmail.com

Rakshit Mathur
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India
rakshitmathur01@gmail.com

Shardul Parag Rawal
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India
shardulrawal02@gmail.com

Jhanvi Vashishth
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India
vashishthjhanvi@gmail.com

***Abstract*—DUSt3R and MASt3R recover dense scene geometry from 2D images without camera poses or calibration, but a deployable 2D-to-3D system also needs ingestion, frame selection, synchronization, geometry refinement, and application-specific delivery to work together. We present z-shift, a four-stage pipeline of replaceable stages exchanging only serializable artifacts. Ingestion unifies image folders, videos, and synchronized multi-camera captures under a common spatial schema, and buffers live WebSocket frames on a separate track. Reconstruction uses MASt3R with optional Truncated Signed Distance Function (TSDF) fusion and a point-map fallback, refinement applies a fixed sequence of operations, and delivery routes GLB or point-cloud output under a compatibility contract. We evaluate with a constrained benchmark suite, one real capture traced through every stage, and five DTU scans scored against ground truth. Motion-ranked frame retention degrades sliding-window pairing locality 6.7-fold unless capture order is restored, which costs nothing. Refinement cost depends more on disconnected mesh components than on triangle count, staying nearly flat below about 100 components and reaching 20-fold at 1,000; batched labeling removes that dependence. Repeat runs were byte-identical on each of two GPUs. Against DTU ground truth, ten views score 3.3–8.6 mm overall, and neither 49 views, restored capture order, nor refinement improved scores consistently. Object isolation preserves F-score on four of five scans while removing most of the mesh, and DUSt3R runs as a second backend with every other stage unchanged. Reconstruction from a single image or a live stream, which the pipeline rejects, real-time delivery, and efficient large-scale mesh processing remain future work.**

***Index Terms*—2D-to-3D generation, 3D reconstruction, computer vision, MASt3R, mesh processing, multi-view stereo, point clouds, spatial computing**

I. Introduction

Recovering 3D structure from 2D imagery has progressed from image-based reconstruction [2] and Structure-from-Motion [3], through learning-based multi-view methods [4], [5], to pose-free dense regression with DUSt3R [6] and MASt3R [7]. An operational system also requires input normalization, frame selection, synchronization, geometry refinement, and delivery tailored to the application.

z-shift is a four-stage process whose individually replaceable stages exchange serializable artifacts. Ingestion unites image folders, videos, and synchronized captures, and buffers live streams, under a uniform spatial representation; reconstruction accepts the multi-view inputs and rejects single images and live streams.

Our evaluation targets the stage interfaces: how frame ordering affects pairing, how fragmentation drives refinement cost, whether repeat runs are reproducible, and how the assembled pipeline scores against DTU ground truth [19].

II. Related Work

Several established research directions in the field of 3D reconstruction provide well-developed techniques for geometric reconstruction from multiple images. Recent surveys of dense 3D reconstruction describe the geometric foundations and datasets used for image-based reconstruction [2], while Herrera-Granda et al. [3] review Structure-from-Motion, visual odometry, and SLAM methods for 3D reconstruction. Wang et al. [4] survey deep learning-based approaches for 3D reconstruction from multiple images, while Wang et al. [5] consider learning-based multi-view stereo for dense reconstruction. Machine learning has enabled many contributions in the field of 3D reconstruction with neural representations like NeRF [16], Instant-NGP [20], 3D Gaussian Splatting [21], and others [13], [14], [22]. DUSt3R [6] and MASt3R [7] enable dense geometry estimation with minimal reliance on conventional pose estimation techniques. The above-mentioned contributions mostly deal with reconstruction problems themselves, while the organization of other stages depends on the application at hand.

In general, reconstructed geometry needs some processing to become useful as a resource. Yang et al. [1] propose a

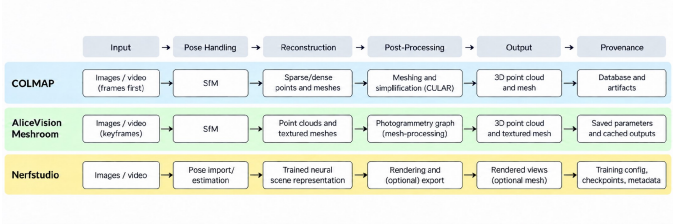


Fig. 1. Comparison of representative reconstruction systems.

feature-driven mesh denoising technique aimed at reducing noise while preserving important geometric features, while Maruani et al. [8] present a neural mesh representation based on Quadric Error Metrics. Surface fusion of reconstructed geometry in volumetric representations is shown by Hu et al. [9] through color truncated signed distance field fusion, while Johari et al. [23] reconstruct dense surface geometry using a hybrid representation of signed distance fields. Kazhdan and Hoppe [24] present a distributed approach to Poisson surface reconstruction for producing surface geometry from oriented points. Even though the above-mentioned works address geometry processing tasks, they use different representations of input geometry, assumptions, and parameters.

On a system level, AliceVision Meshroom [12] presents an open-source photogrammetry system, while Nerfstudio [25] gives a flexible framework for neural field geometry reconstruction and visualization. Open3D [18] and Kaolin [17] provide reusable 3D processing and learning components instead of a fixed ingestion-to-delivery workflow. Evaluation datasets such as DTU [19] and CO3D [15] further support research in reconstruction with both controlled and real-world capturing conditions. All these works present strong individual components and frameworks that vary greatly in their input normalization, stage connections, and output processing for downstream usage. Fig. 1 shows how the boundaries between stages of these works differ from those proposed in our paper, summarized from the documentation of COLMAP [26], Meshroom [27], and Nerfstudio [28].

III. Research Gap and Problem Statement

A stage may comply with its own specification yet violate an assumption made by the following stage. It may also introduce a cost profile that does not appear when stages are measured separately. This paper therefore focuses on the correctness and cost of the interfaces between stages, rather than the validity or cost of the model used in the reconstruction stage.

IV. System Architecture

Four stages (Fig. 2) communicate only through serializable artifacts: a unified schema, a reconstruction job, a mesh file, and a set of manifests.

A. Media Ingestion

An HTTP/WebSocket gateway ingests single images, image folders, videos, video folders, and live WebSocket frames. MIME type and extension select the processing track, and mixed or unrecognized payloads are rejected outright. On the batch track, images are resized and videos are sampled by a motion-adaptive sampler. For a video folder, a multi-source synchronizer estimates a per-source clock offset by matching motion signatures, then aligns frames by nearest timestamp. On the live track, frames enter a bounded ring buffer. Both tracks converge into one serializable schema, which decouples ingestion from reconstruction and makes an input replayable.

B. Multi-View Reconstruction

An image folder runs as multi-view, a single video as a video-sequence, and a folder of videos as synchronized-views. Alignment needs at least two views of one subject, so single images and live streams are rejected. Above a configurable frame budget, video jobs retain the highest-motion frames, and synchronized captures are budgeted by group. Pairing uses a complete scene graph or a sliding window. Frames proceed to pairing in motion-rank order, not capture order, an interaction evaluated in Section V-C.

Before alignment, the pipeline selects a device, seeds its generators if a seed is supplied, and derives approximate intrinsics from the EXIF focal length [10]. MASt3R then performs sparse global alignment using its official implementation [11], with results cached to disk. TSDF fusion may precede export; if it is disabled or fails, the pipeline falls back to a dense point map and records the event. The output is a colored GLB mesh and a confidence-masked point cloud, and the run manifest records the model, device, resolution, pairing strategy, thresholds, and seed. The stage sits behind a backend interface selected per job, so another model can replace MASt3R (Section V-F).

C. Mesh Refinement

A fixed procedure is applied to every mesh. Object mode then keeps the largest connected component, while room mode merges components above a minimum cell count. Boundary holes are closed, Taubin smoothing reduces noise without Laplacian shrinkage, finalization merges, decimates, and recomputes normals, and vertex colors are transferred by nearest point. A non-watertight result is reported as a warning, not an error.

D. Application-Specific Deliverables

Each use case declares the source types it can consume, and every other combination is rejected before export; all 24 combinations route or reject correctly (Table II). Editing emits a GLB and viewing a colored PLY point cloud rather than Gaussian Splatting [21]. The live route rejects requests.

V. Results and Discussion

A. Computational Performance

The as-submitted refinement splits the mesh into bodies on the CPU. Timings here are means over three repeats (five for Table I) on an Intel Core i7-14650HX laptop (24 logical cores) under WSL2. On a single connected mesh, a log-log fit over

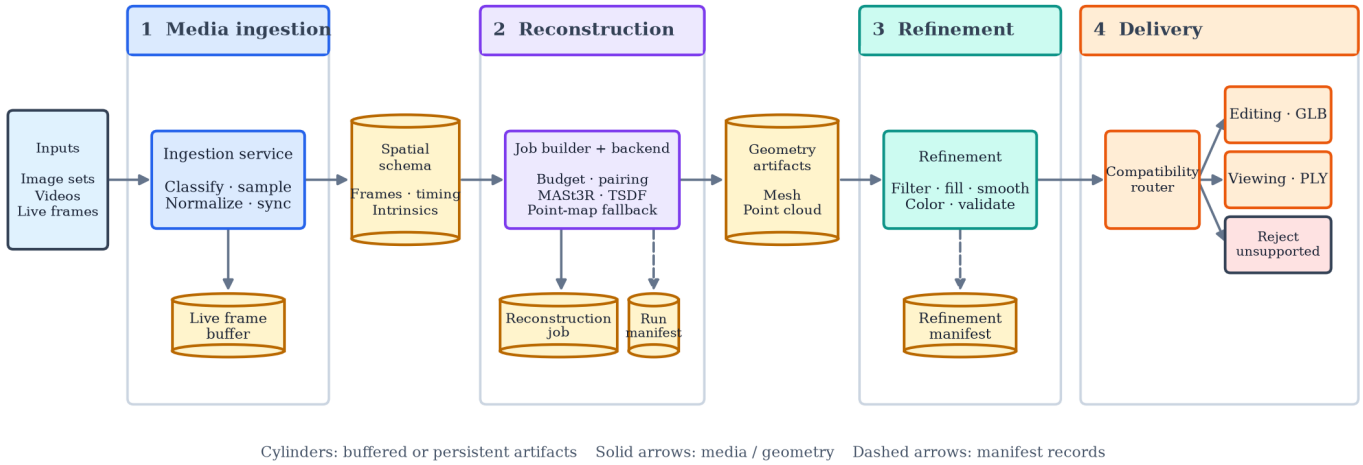


Fig. 2. System architecture.

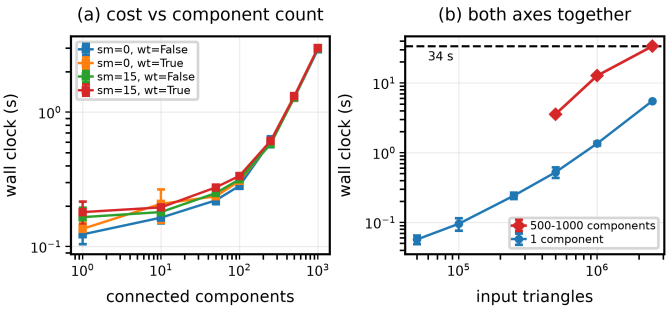


Fig. 3. Refinement time vs. (a) components and (b) components and triangles (SD over 3 repeats; dashed: slowest case, 34 s).

TABLE I
REFINEMENT COST BY STAGE (1M TRI. IN 1 COMPONENT; 96K IN 1,000)

Stage	Large Mesh	Fragmented Mesh
Component filtering	0.26 s (14.8%)	2.97 s (96.5%)
Hole filling	0.17 s (9.5%)	0.01 s (0.3%)
Smoothing	0.24 s (13.3%)	0.03 s (1.0%)
Finalization	1.00 s (55.8%)	0.04 s (1.3%)
Watertightness check	0.12 s (6.5%)	0.03 s (0.9%)
Total	**1.79 s**	**3.07 s**
Total, batched labeling	*1.55 s*	*0.11 s*

50K to 2.5M triangles gives an exponent of 1.10–1.21 across eight configuration and geometry combinations ($R^2 \geq 0.98$).

Component count is the stronger driver. At a fixed ∼96,000 triangles (Fig. 3(a)), cost grows 20x between 1 and 1,000 components. The curve has a knee: the local exponent is 0.15 from 1 to 100 components (0.15 s to 0.31 s) and 0.98 from 100 to 1,000. Fragmentation therefore behaves as a threshold rather than a gradient. Table I shows why: finalization dominates on a large connected mesh, while on a fragmented one component filtering takes 96.5%, because body splitting runs each component through a separate pass.

This predicts its own fix, which we implemented: one batched connected-component labeling pass on the GPU replaces the per-body loop with identical cell and point counts on every scaling configuration. Cost no longer depends on component count (0.13 s at one component, 0.09 s at 1,000), and the slowest combined-load case falls from 34 s to 4.2 s. The cost belongs to an interface, not a stage, and no benchmark of refinement on a single connected mesh would reveal it.

B. Determinism and Repeat-Run Behavior

Three DTU scenes [19] at 8 views each ran twice under one seed with the cache cleared, with PyTorch's deterministic mode on and off, on a Tesla T4 and on an RTX 5060 laptop GPU (8 GB; WSL2, CUDA 13, PyTorch 2.13). On both, repeats were identical regardless of the mode (Table IV). PyTorch does not guarantee identical results across releases or platforms [29], and output was not compared across the two machines; the recorded manifest sufficed for bit-identical reruns on each tested environment.

C. Component and Integration Correctness

Table II summarizes contract checks on a synthetic, single-body protocol. Distance to the clean ground truth improves as injected corruption is removed (Fig. 4(a)–(b)), which the protocol guarantees by construction. The informative comparison is that Taubin holds mesh volume within 0.16% of the unsmoothed baseline, against 0.51% drift for Laplacian (Fig. 4(c)).

Frame ordering. Motion-ranked retention changes frame order before sliding-window pairing, which assumes capture order; re-sorting costs nothing and recovers 6.7x (Table II). Neither stage is incorrect alone: the defect appears only when they are composed.

TABLE II
CORRECTNESS CHECKS (SYNTHETIC, CPU)

Component	Evaluation	Key Result
Component filtering	0/5/20 injected fragments	All injected fragments removed; object and room mode converge, as expected on a single body
Frame selection / pairing	3 over-budget video schedules	Re-sorting before pairing reduces median pair-index gap 186.3 → 27.7 (6.7x pooled, 4.4–10.2x per schedule)
Synchronization	4,455 trials, 0–640 ms offset, jitter, 2–4 sources	2.14 ms mean absolute offset error; 100% sign accuracy; 99.8% group formation
Motion sampling	Controlled motion windows	1.2–2.4x concentration over uniform sampling; collapses toward 1.0x for low-magnitude motion
Routing	24 source-type / use-case combinations	100% correct routing or rejection, no default fallthrough
Color transfer	Analytic color field	<1/255 mean per-channel error across all settings

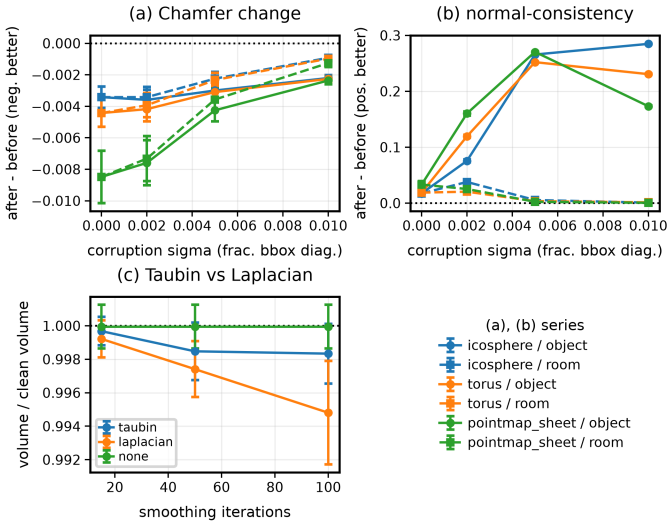


Fig. 4. Refinement vs. corruption: (a) Chamfer, (b) normal consistency, (c) volume retained by smoothing (SEM).

TABLE III
CO3D TEDDYBEAR STAGE TIMINGS

Stage	8 views, as submitted	24 views, current
Sparse alignment	78.2 s (56 pairs)	587.9 s (552 pairs)
Mesh export	8.6 s; 1.84M tri, 63.1% conf. retained	47.2 s; 2.51M tri, 41.5% conf. retained
Reconstruction total	*86.9 s*	*635.0 s*
Support-plane cut	—	3.0 s (69% of faces)
Component filtering	1807.7 s (per body, CPU)	0.33 s (batched, GPU)
Hole filling, smoothing, finalization, color transfer	0.43 s	0.40 s
Watertightness check	0.06 s; not watertight	0.02 s; not watertight
Refinement total	*1808.2 s; 397K cells*	*3.7 s; 204K cells*
Total	**1895.1 s (≈32 min)**	**638.8 s (≈11 min)**

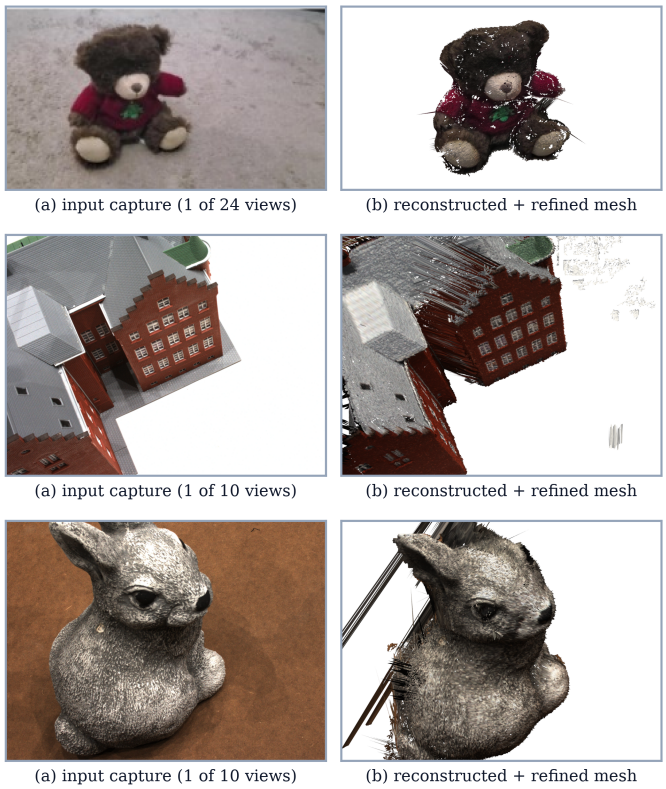


Fig. 5. Input and isolated refined mesh from the estimated camera: teddybear (24 views), DTU scans S1 and S4 (10 views).

D. Illustrative End-to-End Trace

We traced the workflow on a real CO3D *teddybear* capture [15] at eight evenly spaced frames, as submitted, and at 24 (Table III). Each is a single run, which supports neither a timing nor a quality claim.

At eight views, per-body filtering took 99.97% of refinement, as the benchmark predicts far past the knee. Batched labeling returns identical counts on the same mesh in 0.31 s.

Tripling the views exposed a defect the benchmarks could not. Object mode assumes the subject is the largest thing reconstructed, but a real capture also reconstructs the support surface. At 24 views the floor was a separate, larger piece, and the refined mesh was floor alone, while every stage met its own contract. We therefore added an optional object-isolation step, off by default: it removes the dominant RANSAC plane when that plane holds at least 30% of the faces (69% here), then keeps the largest remaining piece and the fragments inside its bounding box (Fig. 5). Holes and silhouette streaks remain; Section V-E measures accuracy.

E. Reconstruction Accuracy on DTU

We score five scans against DTU's structured-light ground truth [19] using DTU's evaluation, with overall error the mean of accuracy and completeness, distances over 20 mm ignored,

TABLE IV
QUALITY AND REPRODUCIBILITY BENCHMARKS

Evaluation	Headline Result
Controlled mesh corruption; 648 paired refinement trials	Object mode: Chamfer -64.2%, F@1% $+3.79$ points, normal consistency $+0.158$; all injected fragments removed.
Five real DTU scans (10 views); raw vs. refined mesh	Object mode: overall error $+0.65$ mm, F@2 mm -0.020. Room mode: $+0.44$ mm, -0.021. The synthetic gain above does not carry over (Section V-E).
Three DTU scenes (8 views); six paired repeat comparisons on Tesla T4	All pairs have zero Chamfer/H95, identical point counts and byte-identical GLBs; deterministic kernels add 21.3% time.
Same three scenes (8 views); six paired repeat comparisons on RTX 5060 laptop GPU (WSL2, CUDA 13)	All pairs have zero Chamfer/H95, identical point counts and byte-identical GLBs; deterministic kernels add 38.8% time.

TABLE V
DTU OVERALL ERROR (MM) / F@2 MM (†: POSE FAILURE)

Scan	10 views	49 views	V1	V2	V3
S1	5.99/.126	4.28/.331	2.87/.549	4.60/.340	6.17/.261
S2	8.59/.112†	7.99/.090†	5.66/.247†	6.05/.172†	9.16/.069†
S3	5.99/.171	6.36/.106	5.78/.270†	7.20/.080†	4.91/.429
S4	3.30/.411	4.56/.340	5.64/.232	10.11/.057†	6.91/.183†
S5	5.39/.231	7.63/.128†	7.80/.135†	6.56/.146†	7.72/.135†
Mean	**5.85/.210**	**6.16/.199**	**5.55/.287**	**6.90/.159**	**6.97/.215**

and F-score at $\tau = 2$ mm. Runs use 512-pixel input on the RTX 5060.

MVS methods are normally given DTU's cameras; MASt3R estimates its own. Each reconstruction is therefore placed in DTU's frame by a similarity fit from its estimated camera centers to the calibrated ones. The fit is trimmed: a camera whose residual exceeds both 10 mm and three times the median is left out. A run is flagged as a pose failure if the remaining RMSE exceeds 10 mm or more than 20% of its cameras are left out.

Table V gives the results; means include flagged runs. Ten spread views with complete pairing score 3.30–8.59 mm overall in 157 s on average. Forty-nine views, on a sliding window of 9, take 7.4x as long and do not reliably improve on this: overall error is lower on two of five scans and F-score higher only on S1. Much of the error is placement: at ten views, rigid ICP against the ground truth, which a deployment could not run, lowers mean overall error to 3.64 mm and raises mean F-score to 0.422. Overall error can also flatter a failed run, since it ignores distant points: S2 at 49 views drops 37% of its reconstructed points and 53% of the ground truth. Pose failures affect 3 of these 10 runs, including both on S2.

Columns V1–V3 test frame ordering under a 40-frame budget, with DTU's 49 views standing in for a video: V1 is motion-rank order as implemented, V2 capture order, and V3 a uniform subsample. The three seeds reach only TSDF sampling, so the sample is five scans rather than 15 runs.

TABLE VI
FACES KEPT / F@2 MM ON DTU MESHES

Scan	Raw	Object	Object + isolation
S1	1.00/.165	.26/.138	.43/.164
S2	1.00/.152	.18/.097	.11/.002
S3	1.00/.239	.19/.154	.39/.241
S4	1.00/.318	.11/.378	.33/.336
S5	1.00/.194	.14/.198	.15/.217
Mean	**1.00/.213**	**.18/.193**	**.28/.192**

Restoring capture order showed no benefit, as V2 beats V1 on one scan of five. Nor does this show that motion-rank order is better: depending on whether the fit is trimmed and ICP applied, V1 wins between two and five scans on overall error, even five of five would be only $p = 0.06$ on a two-sided sign test, the trim was chosen post hoc, and 10 of the 15 cells are pose failures.

Sampling mesh surfaces at ten views with isolation off, refinement lowers scores on these real scans (Table IV), and completeness worsens on every scan in object mode. Refinement removes the damage we inject in the synthetic benchmark, but that does not make a real reconstruction more accurate.

F. Further Evaluation

Refinement on real meshes. On the raw meshes (1.3–2.6M cells), batched labeling takes 0.30–0.84 s (SD at most 0.09 s over five repeats). The per-body filter finished within a 30-min budget only on S2, in 1,259 s with the same output, about 4,000x slower. Table VI shows what each variant keeps, and Fig. 5 shows two of the five scans. Object isolation cuts the support plane (32–60% of faces), keeps 15–43% of faces on four scans, and stays within 0.001 of or above the raw mesh's F-score on them (mean 0.239 against 0.229 raw), while overall error is level on two and up to 0.89 mm higher on two. On S2, a pose failure throughout Table V, its score collapses. The plain largest-component rule lowers F-score on three of five scans.

CO3D against its reference cloud. CO3D's cloud is a COLMAP reconstruction rather than independent ground truth, and the single-sequence release has no cameras, so we register each result to it by a fixed scale and rigid ICP (residual 1.4–1.5% of the bounding-box diagonal) and treat it as a pseudo-reference. At F@2% of the diagonal, the refined, isolated mesh reaches 0.745 at eight views and 0.769 at 24, against 0.586 and 0.502 for the raw point cloud. Registration favors any well-shaped surface, so these figures compare variants, not methods.

Synchronization on real images. The production motion score is computed on real DTU frames and a second camera's on its own perturbed pixels, with clocks shifted by 0–640 ms over 1,100 trials. DTU's cameras carry no timestamps, so clocks are injected, and this is not a recording with true clocks. Median absolute error is 12–30 ms per scan against a 40 ms frame interval, the sign is correct in 84–97% of nonzero-offset trials, and grouping reaches 83–90%. Mean error (31–145 ms)

is larger because a minority of trials fail by much more than the median.

A second backend. DUSt3R [6] ran through the backend interface on the ten views of Table V. On all five scans the stage wrote the same artifacts and refinement consumed the mesh unchanged. The swap also moves the measurement: DUSt3R reaches mean F-score 0.069 and 8.3 mm overall against 0.210 and 5.85 mm for MASt3R, with all five scans flagged as pose failures, in 64 s per scan against 157 s. The reconstruction stage can thus be replaced and compared under one protocol; we make no such claim for other stages.

VI. Conclusions

z-shift is a system architecture of replaceable stages exchanging explicit artifacts, not a new reconstruction network. Frame ordering that is correct at selection breaks pairing locality at the next stage, and mesh fragmentation governs refinement cost as a threshold, which batched labeling removes. None of this surfaces from a benchmark of one stage. Nor does a synthetic gain imply a real one: against DTU ground truth, the pipeline scores 3.3–8.6 mm overall at ten views, much of it placement error, and restoring capture order, adding views, and refinement each failed to improve scores consistently.

VII. Limitations and Future Work

Real-time delivery is incomplete: WebSocket ingestion works, but RTSP/WebRTC and the live output route are unimplemented, and viewing exports a colored point cloud. Most of refinement stays on the CPU. The ground-truth benchmark covers five scans of one dataset at 512-pixel input, camera pose fails on 3 of its 10 view-count runs and 10 of its 15 frame-ordering cells, and no video with ground truth was tested.

No classical baseline ran under the same protocol, and published MVS scores, which are given DTU's cameras, are not comparable. More robust camera estimation and comparison against classical MVS baselines under one protocol are the priorities for future work.

References

[1] J. Yang, C. Wang, H. Hou, M. Wang, and X. Li, "Feature-driven variational mesh denoising," *IEEE/CAA J. Autom. Sinica*, vol. 12, no. 5, pp. 913–924, May 2025, doi: 10.1109/JAS.2024.124923.

[2] Y. Lee, "Three-dimensional dense reconstruction: A review of algorithms and datasets," *Sensors*, vol. 24, no. 18, p. 5861, 2024, doi: 10.3390/s24185861.

[3] E. P. Herrera-Granda, J. C. Torres-Cantero, and D. H. Peluffo-Ordóñez, "Monocular visual SLAM, visual odometry, and structure from motion methods applied to 3D reconstruction: A comprehensive survey," *Heliyon*, vol. 10, no. 18, p. e37356, 2024, doi: 10.1016/j.heliyon.2024.e37356.

[4] C. Wang et al., "Deep learning-based 3D reconstruction from multiple images: A survey," *Neurocomputing*, vol. 597, p. 128018, 2024, doi: 10.1016/j.neucom.2024.128018.

[5] F. Wang et al., "Learning-based multi-view stereo: A survey," 2024, arXiv:2408.15235.

[6] S. Wang, V. Leroy, Y. Cabon, B. Chidlovskii, and J. Revaud, "DUSt3R: Geometric 3D vision made easy," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024.

[7] V. Leroy, Y. Cabon, and J. Revaud, "Grounding image matching in 3D with MASt3R," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2024.

[8] N. Maruani, M. Ovsjanikov, P. Alliez, and M. Desbrun, "PoNQ: A neural QEM-based mesh representation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024, pp. 3647–3657.

[9] Q. Hu et al., "RGBTSDF: An efficient and simple method for color truncated signed distance field (TSDF) volume fusion based on RGB-D images," *Remote Sens.*, vol. 16, no. 17, p. 3188, 2024, doi: 10.3390/rs16173188.

[10] *Exchangeable Image File Format for Digital Still Cameras: Exif Version 3.0*, CIPA DC-008-2024, Camera & Imaging Products Association, 2024.

[11] NAVER Corp., "MASt3R: Grounding image matching in 3D," software repository, 2024. [Online]. Available: https://github.com/naver/mast3r (accessed Sep. 11, 2026).

[12] C. Griwodz et al., "AliceVision Meshroom: An open-source 3D reconstruction pipeline," in *Proc. 12th ACM Multimedia Syst. Conf. (MMSys)*, 2021, pp. 241–247.

[13] A. Chen et al., "MVSNeRF: Fast generalizable radiance field reconstruction from multi-view stereo," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2021, pp. 14124–14133.

[14] A. Yu et al., "pixelNeRF: Neural radiance fields from one or few images," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2021, pp. 4578–4587.

[15] J. Reizenstein et al., "Common Objects in 3D: Large-scale learning and evaluation of real-life 3D category reconstruction," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2021, pp. 10901–10911.

[16] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "NeRF: Representing scenes as neural radiance fields for view synthesis," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2020, pp. 405–421.

[17] K. M. Jatavallabhula et al., "Kaolin: A PyTorch library for accelerating 3D deep learning research," 2019, arXiv:1911.05063.

[18] Q.-Y. Zhou, J. Park, and V. Koltun, "Open3D: A modern library for 3D data processing," 2018, arXiv:1801.09847.

[19] R. Jensen, A. Dahl, G. Vogiatzis, E. Tola, and H. Aanæs, "Large scale multi-view stereopsis evaluation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2014, pp. 406–413.

[20] T. Müller, A. Evans, C. Schied, and A. Keller, "Instant neural graphics primitives with a multiresolution hash encoding," *ACM Trans. Graph.*, vol. 41, no. 4, 2022.

[21] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3D Gaussian splatting for real-time radiance field rendering," *ACM Trans. Graph.*, vol. 42, no. 4, 2023.

[22] J. T. Barron et al., "Mip-NeRF 360: Unbounded anti-aliased neural radiance fields," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2022, pp. 5470–5479.

[23] M. M. Johari, C. Carta, and F. Fleuret, "ESLAM: Efficient dense SLAM system based on hybrid representation of signed distance fields," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2023, pp. 17408–17419, doi: 10.1109/CVPR52729.2023.01670.

[24] M. Kazhdan and H. Hoppe, "Distributed Poisson surface reconstruction," *Comput. Graph. Forum*, vol. 42, no. 6, Art. no. e14925, 2023, doi: 10.1111/cgf.14925.

[25] M. Tancik et al., "Nerfstudio: A modular framework for neural radiance field development," in *Proc. ACM SIGGRAPH Conf.*, 2023.

[26] COLMAP Contributors, "Command-line interface," COLMAP Documentation. [Online]. Available: https://colmap.github.io/cli.html (accessed Sep. 11, 2026).

[27] AliceVision Contributors, "KeyframeSelection," Meshroom Documentation. [Online]. Available: https://meshroom-manual.readthedocs.io/en/latest/feature-documentation/nodes/KeyframeSelection.html (accessed Sep. 11, 2026).

[28] Nerfstudio Contributors, "Using custom data," Nerfstudio Documentation. [Online]. Available: https://docs.nerf.studio/quickstart/custom_dataset.html (accessed Sep. 11, 2026).

[29] PyTorch Contributors, "Reproducibility," PyTorch Documentation. [Online]. Available: https://pytorch.org/docs/stable/notes/randomness.html (accessed Sep. 11, 2026).